ANISOTROPIC STENCIL FILTER COMPUTATIONAL COMPLEXITY ANALYSIS

Naive Line Filter vs. Fused Stencil Formulation

Luke Bagan, Jack Vaught, Yi Wang
Department of Mechanical Engineering
University of South Carolina

OUTLINE

1. Background
2. Naive Line Filter Cost
3. Fused Stencil Formulation
4. ASF Operation Count
5. Direct Comparison
6. Benchmarks
7. Summary

BACKGROUND

Polynomial fitting for gradient estimation

LOCAL POLYNOMIAL GRADIENT ESTIMATION

Model local intensity with a 2D Taylor expansion of order d:

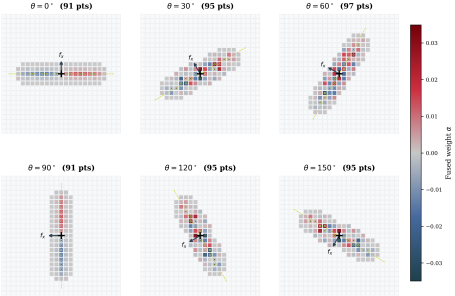
$$f(x_i, y_i) \approx f^0 + f_x x_i + f_y y_i + \frac{f_{xx}}{2} x_i^2 + \cdots$$

$M = \frac{(d+1)(d+2)}{2}$ unknowns $\quad (d{=}4 \rightarrow M{=}15)$

- Gather N_p circular neighbors $\rightarrow$ system $\mathbf{Az} = \mathbf{b}$
- Solve via pseudoinverse: $\hat{\mathbf{z}} = \mathbf{A}^+\mathbf{b}$
- Normal derivative $=$ row 1 of $\mathbf{A}^+ \cdot$ intensities

Key Property

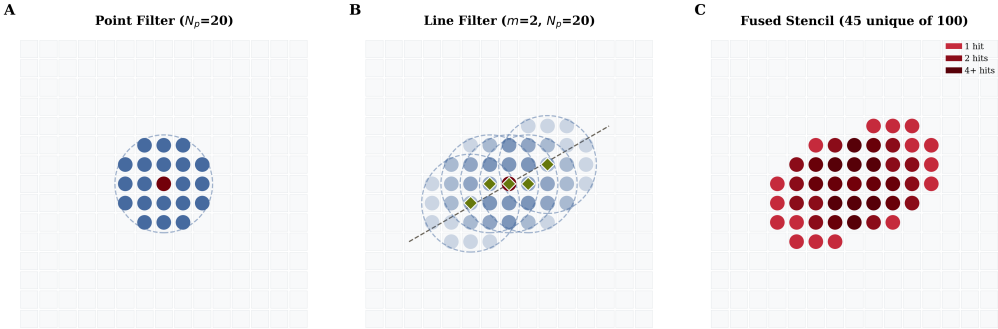
$\mathbf{A}^+$ depends only on geometry and θ $\rightarrow$



Fused stencil weights at 6 orientations.

Red = positive, blue = negative weight.

NEIGHBORHOOD STRATEGIES: POINT, LINE, AND FUSED



(A) Point Filter
$N_p = 15$ circular neighbors around one pixel

(B) Line Filter
$(2m+1)$ virtual points, heavily overlapping

(C) Fused Stencil
Deduplicated — 37 unique of 75 total positions

LINE EXTENSION FOR ROBUSTNESS

Evaluate the point filter at $(2m+1)$ virtual positions along each candidate edge direction:

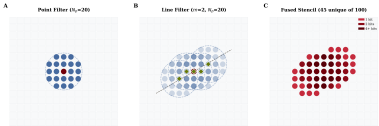
$$R_k = \sum_{j=-m}^{m} w_j \cdot \hat{f}_x^{(j,k)}, \qquad w_j = \exp\left(-\frac{j^2}{2\sigma^2}\right), \quad \sigma = m/2$$

Benefits:

- Wider spatial context $\rightarrow$ noise robustness
- Coherence along the edge tangent $\rightarrow$ bridges gaps

Problem:

- $(2m+1)$ independent gather-matmul ops per orientation per pixel
- Massive memory: $L \times N_p$ intermediate values



Panel B shows 5 overlapping neighborhoods along the line

NAIVE LINE FILTER COST

Per-pixel operation count for the naive approach

NAIVE LINE FILTER: PSEUDOCODE

```
for θ in orientations: # Ns = 18
  for j = -m to m: # L = 15
    for i = 1 to Np: # Np = 100
      READ pixel(j, i) # 1 mem read
      MUL by p_i^(k) # 1 multiply
    SUM Np values # 99 adds
  MUL by Gaussian wj # 15 muls
  SUM over line # 14 adds
  ABS(Rθ)
pick max over θ
```

Per orientation ($m=7$, $N_p=100$):

- Memory reads: $15 \times 100 = \mathbf{1{,}500}$
- Multiplies: $15 \times 100 + 15 = \mathbf{1{,}515}$
- Additions: $15 \times 99 + 14 = \mathbf{1{,}499}$

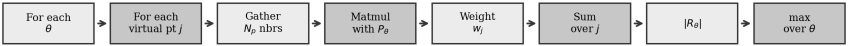
Per pixel (18 orientations):

- Reads: 27,000
- Multiplies: 27,270
- Additions: 26,982
- Total: ~81,252 ops

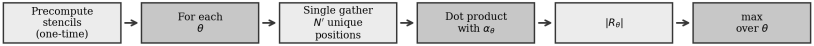
Complexity

$\mathcal{O}(N_s \times (2m+1) \times N_p)$ per pixel $= \mathcal{O}(N_s \cdot L \cdot N_p)$

NAIVE LF: COMPUTATION FLOW



(A) Naive Line Filter compute process. Complexity: $O(N_s \times L \times N_p)$ per pixel



(B) Anisotropic Stencil Filter compute process. Complexity: $O(N_s \times N')$ per pixel

- Top: Naive LF requires nested loops — 8 sequential stages per pixel
- Bottom: Fused ASF collapses to 6 stages, with the inner 5 replaced by one gather + dot product
- Memory: naive allocates 6.2 GB of intermediates vs. 20 MB for ASF

FUSED STENCIL FORMULATION

Collapsing the pipeline into a single dot product

ALGEBRAIC COLLAPSE

The line-extended response is a linear operation on pixel intensities:

$$R_k = \underbrace{\sum_{j=-m}^{m} w_j \sum_{i=1}^{N_p} p_i^{(k)} \cdot f(\text{pixel}_{j,i})}_{\text{Naive: } (2m+1)\times N_p \text{ reads}} = \underbrace{\sum_{\ell=1}^{N'_k} \alpha_{k,\ell} \cdot f(\text{pixel}_\ell)}_{\text{Fused: } N'_k \text{ reads}}$$

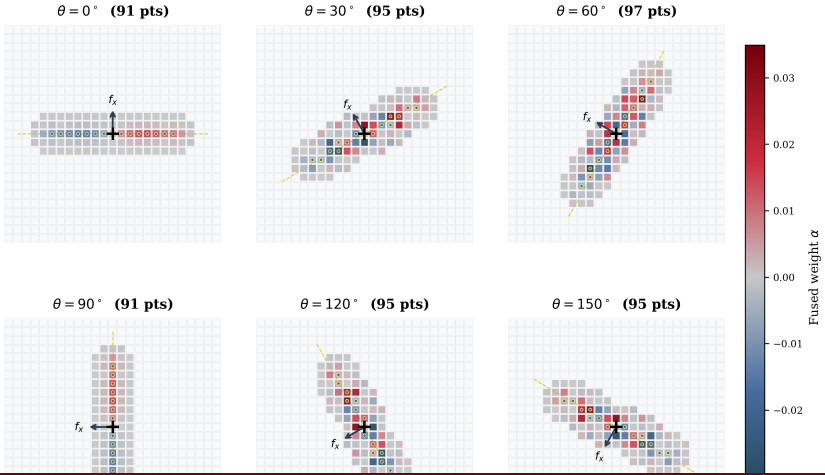
- Many (j, i) pairs map to the same integer pixel after rounding
- Sum their weights $\rightarrow$ single precomputed $\alpha_{k,\ell}$ per unique position
- The entire multi-step pipeline collapses:

Rotate $\rightarrow$ Build $\mathbf{A}$ $\rightarrow$ $\mathbf{A}^+$ $\rightarrow$ Gather $\rightarrow$ MatMul $\rightarrow$ Weight $\rightarrow$ Sum

$$\Downarrow$$

Single Gather + Dot Product with precomputed α_k

STENCIL SHAPES ACROSS ORIENTATIONS



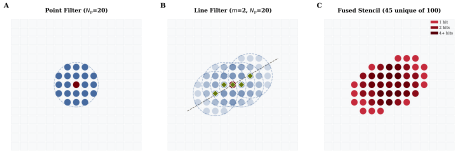
STENCIL DEDUPLICATION

Neighboring virtual positions share most of their circular neighborhoods:

Table: Deduplication ($N_p = 100$, $d = 4$).

m	Raw	Unique	Reduct.	Speedup
1	300	128	57%	2.3×
2	500	152	70%	3.3×
7	1,500	264	82%	5.7×
14	2,900	431	85%	6.7×

- Ratio improves as m increases
- Primary source of computational savings



Panel C: 37 unique of 75 total positions at $m = 2$, $N_p = 15$. 51% overlap eliminated.

ASF OPERATION COUNT

Per-pixel cost after stencil fusion

FUSED ASF: PSEUDOCODE

```
# ONE-TIME precompute:
# αk for each orientation

for θ in orientations: # Ns = 18
  for i = 1 to N′: # N′ ≈ 264
    READ pixel at offset[i] # 1 read
    MUL by α[i] # 1 multiply
  SUM N′ values # 263 adds
  ABS(Rθ)
pick max over θ
```

Per orientation ($m=7$, $N_p=100$):

- Memory reads: 264
- Multiplies: 264
- Additions: 263

Per pixel (18 orientations):

- Reads: 4,752
- Multiplies: 4,752
- Additions: 4,734
- Total: $\sim$14,238 ops

Complexity

$\mathcal{O}(N_s \times N')$ per pixel where $N' \approx 0.18 \times (2m+1) \times N_p$

DIRECT COMPARISON

Naive LF vs. Fused ASF head-to-head

PER-PIXEL OPERATION COMPARISON

Table: Per-pixel cost at $m=7$, $N_p=100$, $N_s=18$, $d=4$.

Metric	Naive LF	Fused ASF	Reduction
Memory reads / orientation	1,500	264	$5.7\times$
Multiplies / orientation	1,515	264	$5.7\times$
Additions / orientation	1,499	263	$5.7\times$
Total ops / pixel	81,252	14,238	$5.7\times$
Runtime matrix ops / pixel	18 matmuls	0	∞

- $5.7\times$ arithmetic reduction from deduplication alone at $m=7$
- Additional gains on GPU from eliminating runtime matrix operations
- At $m=14$: $6.7\times$ arithmetic $\rightarrow$ $24\times$ wall-clock speedup

GPU MEMORY COMPARISON

Naive Line Filter

- Intermediate: $B \times L \times N_p \times 24$ bytes
- Scales with m and N_p
- Peak VRAM: 6,223 MB
- Cannot fit large m on consumer GPUs

Fused ASF (Triton)

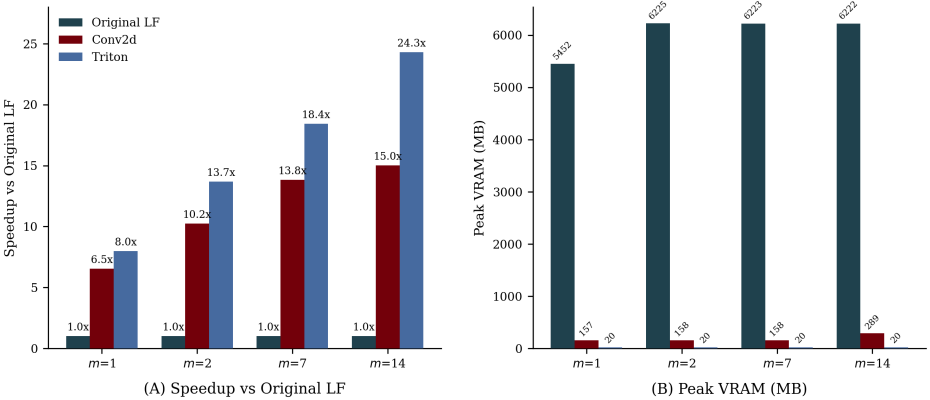
- No intermediate tensors
- Image + output + stencil weights only
- Peak VRAM: 20 MB
- Constant regardless of m

$311\times$ VRAM reduction — 6,223 MB $\rightarrow$ 20 MB

BENCHMARKS

Wall-clock performance on NVIDIA A100

SPEEDUP AND VRAM



- Speedup scales with m due to
- Triton VRAM: constant 20 MB

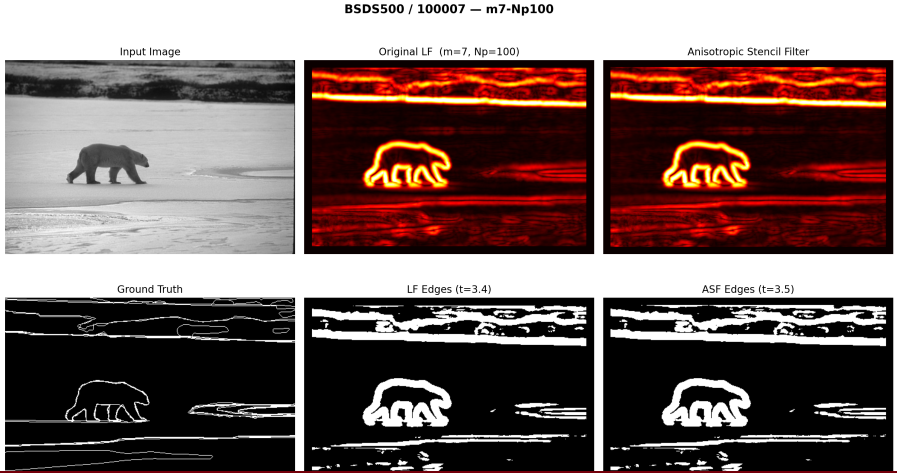
WALL-CLOCK BENCHMARKS

BIPED v1 (1280×720), $N_p = 100$, $N_s = 18$, $d = 4$ — NVIDIA A100-SXM4-40GB:

m	Naive LF (s)	Conv2d (s)	Triton ASF (s)	Speedup
1	0.58	0.089	0.072	8.0×
2	1.91	0.187	0.140	13.7×
7	2.43	0.176	0.132	18.4×
14	8.88	0.591	0.366	24.3×

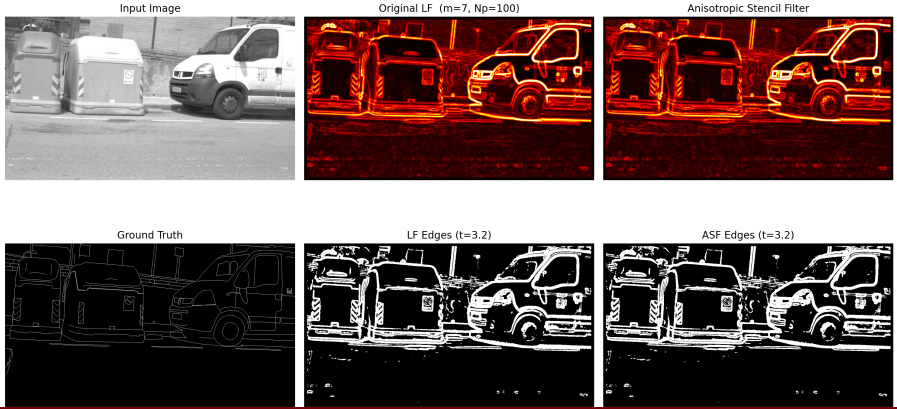
- Additional speedup beyond dedup ratio from eliminating runtime matrix ops
- Triton kernel: 132 ms per 720p image at $m = 7$

VISUAL RESULTS: NAIVE LF VS ASF ARE IDENTICAL



VISUAL RESULTS: BIPED V1

BIPED / RGB_008 — m7-Np100



WHERE DO THE SAVINGS COME FROM?

1. Deduplication

- Overlapping neighborhoods share pixels
- 82% reduction at m=7
- $1{,}500 \rightarrow 264$ reads
- Improves with m

$\rightarrow$ 5.7$\times$ fewer memory accesses

2. Precomputed Weights

- Pseudoinverse, rotation, Gaussian all fused into α_k
- Zero runtime matrix ops
- Eliminates $\mathcal{O}(N_p \times M)$ matmul/orient.

$\rightarrow$ Removes entire matmul stage

3. GPU Kernel Design

- Single gather + dot per orientation
- Vectorized across 128-px blocks
- Coalesced row reads

$\rightarrow$ Constant 20 MB VRAM, fits any GPU

SUMMARY

SUMMARY

The naive line filter is memory-bound and redundant:

- $\mathcal{O}(N_s \times L \times N_p)$ per pixel, most pixel reads duplicated
- $\sim$81,000 total ops/pixel at $m=7$, $N_p=100$, $N_s=18$
- 6.2 GB VRAM for a single 720p image

The fused stencil eliminates all redundancy:

- $\mathcal{O}(N_s \times N')$ per pixel, $N' \approx 0.18 \times L \times N_p$
- $\sim$14,000 total ops/pixel — 5.7$\times$ arithmetic reduction
- Zero runtime matrix operations

GPU results exceed the theoretical arithmetic speedup:

- 18.4$\times$ wall-clock at $m=7$, 24$\times$ at $m=14$
- 311$\times$ VRAM reduction (6.2 GB $\rightarrow$ 20 MB)
- Enables deployment on any GPU, including embedded

THANK YOU!

Questions & Discussion

vaught@email.sc.edu